

rmap

May 22, 2025

1 rmap: An R Package to plot and compare tabular data on customizable maps across scenarios and time

1.1 Maridee Weber

2 Overview

The **rmap** package provides functions to easily create customizable maps and compare results across GCAM scenarios, years, and data classes. The package allows creation of difference maps as well as customizable legends, color palettes, and styles. This notebook demonstrates the primary **rmap** features; more information can be found in the [user guide on GitHub](#).

3 Input structure

rmap requires GCAM output to be in a specific format. Your data should be an R dataframe containing at least the following columns:

- **subRegion**: GCAM region(s)
- **value**: data values

rmap can also utilize the following optional columns:

- **param**: if your data includes multiple queries (e.g., population and GDP)
- **scenario**: if your data includes multiple scenarios
- **class**: if your data contains multiple categories, such as sectors, fuels, land types, etc.
- **year**: if your data contains multiple years
- **units**: if your data contains multiple units

For this tutorial, you will make maps related to population and land use, and will first use **rgcam** to extract this data.

```
[1]: library(rgcam)

# connect to GCAM database
host <- "localhost"
conn <- remoteDBConn('gcamv71_training_basexdb', 'training', 'training', host)
# create project file, using the same one as rgcam here
prj <- addScenario(conn, "rgcam_training", queryFile = "/data/queries.xml",
  ↪scenario = "GCAM", clobber = TRUE)
```

```
prj <- addScenario(conn, "rgcam_training", queryFile = "/data/queries.xml",
  ↪scenario = "GCAM_SSP3", clobber = TRUE)
# retrieve queries
population <- getQuery(prj, "population by region")
land_use <- getQuery(prj, "detailed land allocation")
```

Database scenarios: GCAM, GCAM_SSP3

You can check your dataframes to see if they have the columns required for `rmap`.

```
[2]: head(land_use)
head(population)
```

	Units	scenario	region	landleaf	year	value
	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>
A tibble: 6 × 6	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	1975	0.0075164
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	1990	0.0084019
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	2005	0.0117953
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	2010	0.0141835
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	2015	0.0142252
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	2020	0.0157327
	Units	scenario	region	year	value	
	<chr>	<chr>	<chr>	<dbl>	<dbl>	
A tibble: 6 × 5	thous	GCAM	Africa_Eastern	1975	91890	
	thous	GCAM	Africa_Eastern	1990	145593	
	thous	GCAM	Africa_Eastern	2005	222829	
	thous	GCAM	Africa_Eastern	2010	255333	
	thous	GCAM	Africa_Eastern	2015	292560	
	thous	GCAM	Africa_Eastern	2020	327652	

These two dataframes each have a column containing GCAM regions called **region**, and a column containing values called **value**. The **region** column can be renamed to **subRegion** in each dataframe in order to take advantage of `rmap`'s prebuilt maps.

```
[3]: library(dplyr)
library(tidyr)

population_rmap <- population %>%
  rename(subRegion = region)

land_use_rmap <- land_use %>%
  rename(subRegion = region)
```

Attaching package: 'dplyr'

The following objects are masked from ‘package:stats’:

filter, lag

The following objects are masked from ‘package:base’:

intersect, setdiff, setequal, union

Attaching package: ‘tidyr’

The following object is masked _by_ ‘.GlobalEnv’:

population

```
[4]: colnames(population_rmap)
      colnames(land_use_rmap)
```

1. ‘Units’ 2. ‘scenario’ 3. ‘subRegion’ 4. ‘year’ 5. ‘value’

1. ‘Units’ 2. ‘scenario’ 3. ‘subRegion’ 4. ‘landleaf’ 5. ‘year’ 6. ‘value’

Now the data is ready to use with `rmap`.

4 Using rmap

`rmap` has several options available for customization, but also includes prebuilt maps.

4.1 Example: prebuilt maps

The simplest way to use `rmap` is to let it decide which map to use for you. This is done by passing your dataframe into the `map()` function. `rmap` will see which regions are contained in your **subRegion** column, and select a map based on that.

```
[5]: library(rmap)

# to save time, let's first reduce the number of years in our data frame.
population_rmap_years <- population_rmap %>%
  filter(year == 2025 | year == 2030)

# to produce all map types, use this:
# map(population_rmap_years)

# to produce a specific map, use and modify this
```

```
mapx <- map(population_rmap_years,
            # show = F will prevent every map from being printed, but they will
            ↪ all still be saved to your output folder
            show = F)
# here, specify which maps to show
mapx$map_param_2025_KMEANS
mapx$map_param_2030_KMEANS

# this changes the size of the plot for viewing purposes
options(repr.plot.width = 10, repr.plot.height = 10, repr.plot.res = 100)
```

Starting map...

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Spherical geometry (s2) switched off

More than one pre-loaded map contain the subRegions in the data provided.

Choosing map based on pre-set map ranking.

	map	subRegNum	rank
1	mapGCAMReg32	32	12.0
2	mapGCAMReg32Uruguay	32	12.5

To choose a different map, please assign it in shape directly.

Using map: mapGCAMReg32

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/byYear/
map_param_2025_KMEANS.png

More than one pre-loaded map contain the subRegions in the data provided.

Choosing map based on pre-set map ranking.

	map	subRegNum	rank
1	mapGCAMReg32	32	12.0
2	mapGCAMReg32Uruguay	32	12.5

To choose a different map, please assign it in shape directly.

Using map: mapGCAMReg32

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/byYear/
map_param_2030_KMEANS.png

animation saved in

:/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/anim_param_KMEANS.gif

More than one pre-loaded map contain the subRegions in the data provided.

Choosing map based on pre-set map ranking.

	map	subRegNum	rank
1	mapGCAMReg32	32	12.0
2	mapGCAMReg32Uruguay	32	12.5

To choose a different map, please assign it in shape directly.

Using map: mapGCAMReg32

Figure saved as:

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS.png
More than one pre-loaded map contain the subRegions in the data provided.
Choosing map based on pre-set map ranking.

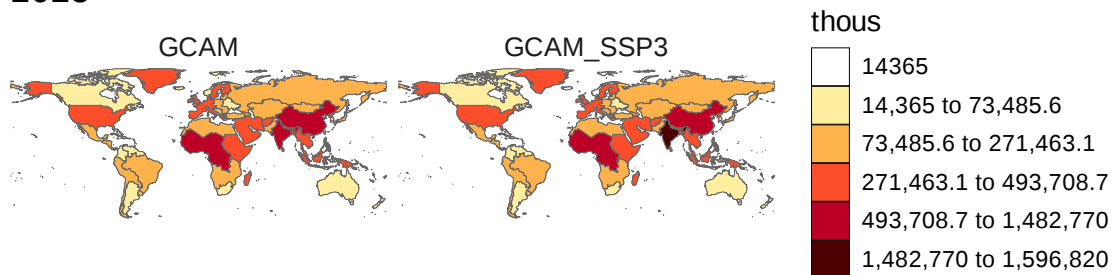
	map	subRegNum	rank
1	mapGCAMReg32	32	12.0
2	mapGCAMReg32Uruguay	32	12.5

To choose a different map, please assign it in shape directly.

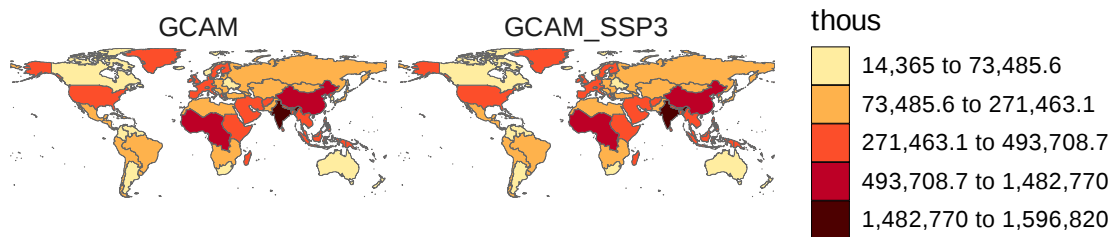
Using map: mapGCAMReg32

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_MEAN_KMEANS.png
map run completed.

2025



2030



Here, `rmap` identified that the `subRegion` column in the `population_rmap` dataframe contains the 32 GCAM region, so it selected `mapGCAMReg32`. Because the dataframe contains multiple years, it made one map for each year, and one map for each scenario.

4.2 Example: custom maps

`rmap` can be used to its full capability by customizing different settings, including the map to use, colors, labels, and others.

4.2.1 Preparing data for custom maps

In order to use different maps in `rmap`, the `subRegion` column needs to contain regional information that `rmap` can recognize. Let's use land use as an example.

```
[6]: head(land_use_rmap)
```

	Units <chr>	scenario <chr>	subRegion <chr>	landleaf <chr>	year <dbl>	value <dbl>
A tibble: 6 × 6	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	1975	0.0075164
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	1990	0.0084019
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	2005	0.0117953
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	2010	0.0141835
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	2015	0.0142252
	thous km2	GCAM	Africa_Eastern	CornC4_AfrCstE_IRR_hi	2020	0.0157327

In its current format, the **landleaf** column contains information about what the land is being used for (CornC4), where the land is (AfrCstE), and what agricultural technologies are being used (IRR_hi). The **subRegion** column contains the aggregate GCAM region, but what if you want a more detailed map of land use? You can modify this dataframe.

This example will show change in land allocated to Forest between our two scenarios.

```
[7]: land_use_forest <- land_use_rmap %>%
  # filter for crop(s) of interest
  filter(grepl("Forest", landleaf)) %>%
  # separate the landleaf column to get the land use region as our subRegion
  separate(landleaf, c("forest1", "forest2", "subRegion"), sep = "_") %>%
  # summarize the dataframe to get aggregate forest values
  group_by(Units, scenario, subRegion, year) %>%
  mutate(value = sum(value)) %>%
  distinct(Units, scenario, subRegion, year, value)

head(land_use_forest)
```

	Units <chr>	scenario <chr>	subRegion <chr>	year <dbl>	value <dbl>
A grouped_df: 6 × 5	thous km2	GCAM	AfrCstE	1975	96.99242
	thous km2	GCAM	AfrCstE	1990	92.97153
	thous km2	GCAM	AfrCstE	2005	89.74967
	thous km2	GCAM	AfrCstE	2010	85.40851
	thous km2	GCAM	AfrCstE	2015	77.55395
	thous km2	GCAM	AfrCstE	2020	77.08776

For more detailed maps, like those using GCAM Basins, sometimes an extra step is needed if your query has abbreviated basin names. To convert these into the names associated with the **mapGCAMBasin** basin names, the **mapping_gcambasins** file provided in **rmap** can be used.

```
[8]: land_use_forest_long <- land_use_forest %>%
  # join in the mapping file
  left_join(mapping_gcambasins, by="subRegion") %>%
  mutate(subRegion=case_when(!is.na(subRegionMap)~subRegionMap,
                             TRUE~subRegion)) %>%
  select(-subRegionMap)
```

```
head(land_use_forest_long)
```

	Units <chr>	scenario <chr>	subRegion <chr>	year <dbl>	value <dbl>
A grouped_df: 6 × 5	thous km2	GCAM	Africa_East_Central_Coast	1975	96.99242
	thous km2	GCAM	Africa_East_Central_Coast	1990	92.97153
	thous km2	GCAM	Africa_East_Central_Coast	2005	89.74967
	thous km2	GCAM	Africa_East_Central_Coast	2010	85.40851
	thous km2	GCAM	Africa_East_Central_Coast	2015	77.55395
	thous km2	GCAM	Africa_East_Central_Coast	2020	77.08776

Now you have long names ready to use by `rmap`.

4.2.2 Selecting a map

`rmap` has numerous preloaded maps - a comprehensive list can be found [here](#). In this example, you can use `mapGCAMBasins` in the `underLayer` argument. To reduce run time, let's first filter the dataframe for a year of interest, otherwise `rmap` will produce figures for all years.

```
[9]: # filter for 2100
land_use_forest_2100 <- land_use_forest_long %>%
  filter(year == 2100)

mapx <- map(land_use_forest_2100,
  show = F,
  underLayer = "mapGCAMBasins")

mapx$map_param_KMEANS

options(repr.plot.width = 10, repr.plot.height = 10, repr.plot.res = 100)
```

Starting map...

Map data table written to

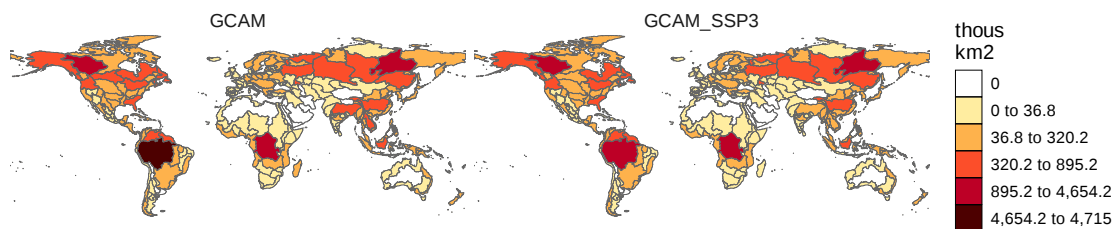
/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as:

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS.png

map run completed.



Here, you see the forested land area at a more detailed spatial scale. To see the difference between two scenarios, use the `scenRef` argument. In this case, the “Reference” scenario is called **GCAM**, while **GCAM_SSP3** represents an alternative scenario.

```
[10]: mapx <- map(land_use_forest_2100,
                 show = F,
                 underLayer = "mapGCAMBasins",
                 scenRef = "GCAM")

# Difference plots automatically produce absolute difference and percent
  ↳ difference
mapx$map_param_KMEANS_DiffAbs
mapx$map_param_KMEANS_DiffPrct
```

```
options(repr.plot.width = 10, repr.plot.height = 10, repr.plot.res = 100)
```

Starting map...

Running difference against all available scenarios:

GCAM_SSP3

Ref scenario chosen for param: param is GCAM

Diff scenarios chosen for param: param are GCAM_SSP3

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as:

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS.png

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS_DiffAbs.png

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

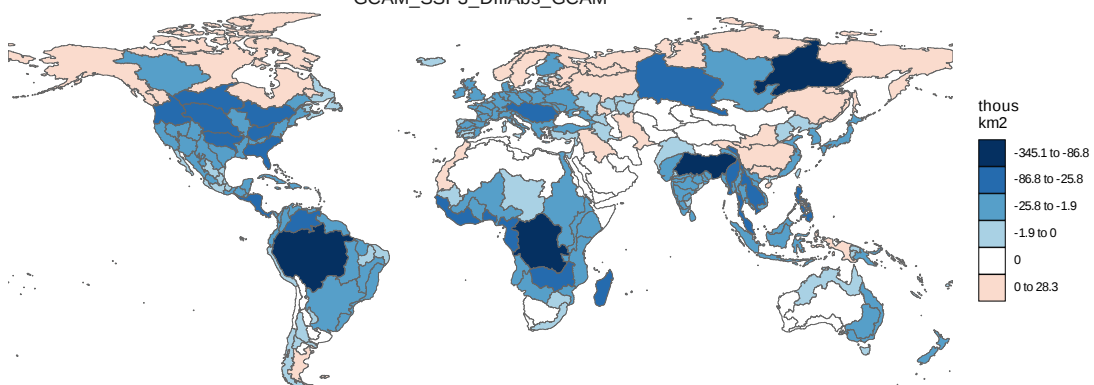
Using map: mapGCAMBasins

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS_DiffPrcnt.png

map run completed.

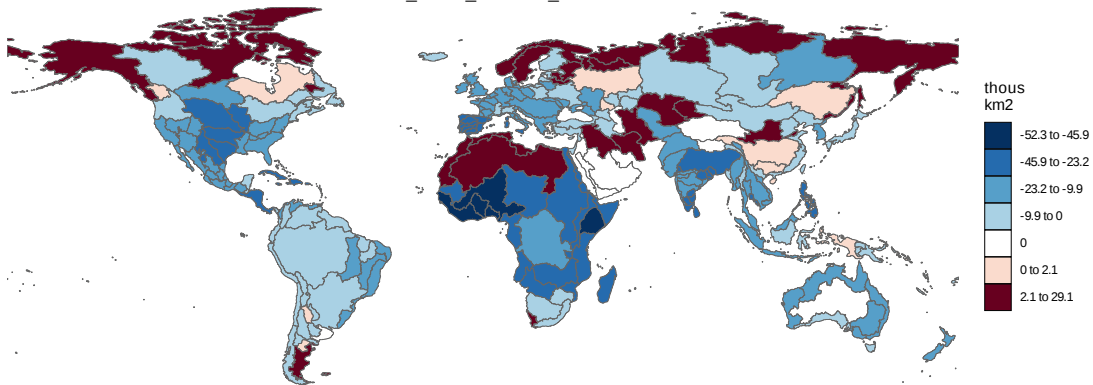
GCAM_SSP3 diffAbs GCAM

GCAM_SSP3_DiffAbs_GCAM



GCAM_SSP3 diffPrnt GCAM

GCAM_SSP3_DiffPrnt_GCAM



4.2.3 Adjusting map features

Legend type There are four types of legend options in `rmap` that can be chosen using the `legendType` argument: “kmeans”, “continuous”, “pretty”, and “fixed”. The default is “kmeans”. Users can set `legendType = "all"` or any combination of `legendType = c("kmeans", "pretty")`.

```
[11]: mapx <- map(land_use_forest_2100,  
  show = F,  
  underLayer = "mapGCAMBasins",  
  scenRef = "GCAM",  
  legendType = "pretty")
```

```
mapx$map_param_PRETTY_DiffPrnt
```

```
options(repr.plot.width = 10, repr.plot.height = 10, repr.plot.res = 100)
```

Starting map...

Running difference against all available scenarios:

GCAM_SSP3

Ref scenario chosen for param: param is GCAM

Diff scenarios chosen for param: param are GCAM_SSP3

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as:

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_PRETTY.png

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_PRETTY_DiffAbs.png

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

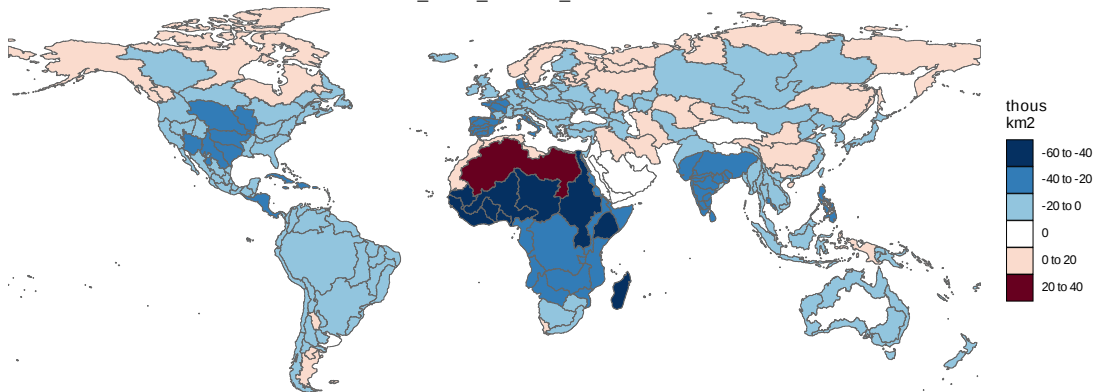
Using map: mapGCAMBasins

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_PRETTY_DiffPrcnt.png

map run completed.

GCAM_SSP3 diffPrnt GCAM

GCAM_SSP3_DiffPrnt_GCAM



Color scheme rmap supports any [R color palette](#), or a custom palette.

```
[12]: mapx <- map(land_use_forest_2100,
  show = F,
  underLayer = "mapGCAMBasins",
  scenRef = "GCAM",
  legendType = "kmeans",
  palette = "pal_green",
  paletteDiff = "pal_div_BrGn")

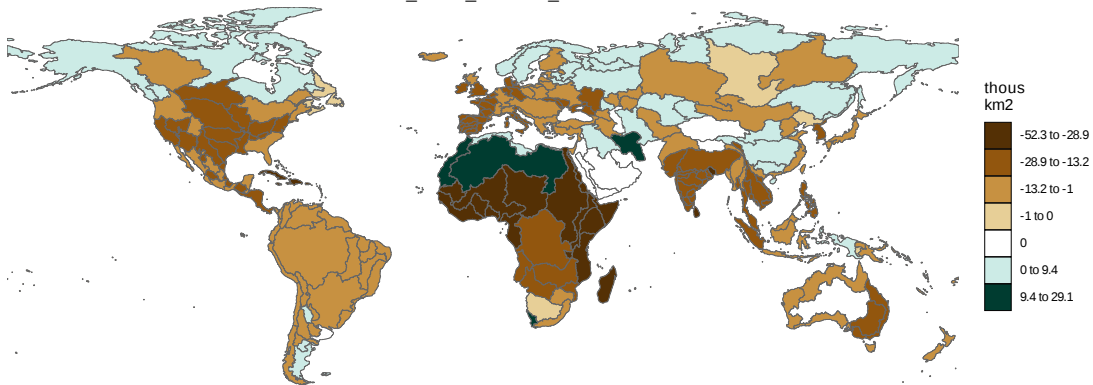
mapx$map_param_KMEANS_DiffPrnt

options(repr.plot.width = 10, repr.plot.height = 10, repr.plot.res = 100)
```

Starting map...
Running difference against all available scenarios:
GCAM_SSP3
Ref scenario chosen for param: param is GCAM
Diff scenarios chosen for param: param are GCAM_SSP3
Map data table written to
/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv
Using map: mapGCAMBasins
Figure saved as:
/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS.png
Map data table written to
/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv
Using map: mapGCAMBasins
Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS_DiffAbs.png
Map data table written to
/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv
Using map: mapGCAMBasins
Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS_DiffPrct.png
map run completed.

GCAM_SSP3 diffPrct GCAM

GCAM_SSP3_DiffPrct_GCAM



Line colors You can also change the color of the regional outlines with the `color` argument.

```
[13]: mapx <- map(land_use_forest_2100,
  show = F,
  underLayer = "mapGCAMBasins",
  scenRef = "GCAM",
  legendType = "kmeans",
  palette = "pal_green",
  paletteDiff = "pal_div_BrGn",
  color = "purple")
```

```
mapx$map_param_KMEANS_DiffPrct
```

```
options(repr.plot.width = 10, repr.plot.height = 10, repr.plot.res = 100)
```

Starting map...

Running difference against all available scenarios:

GCAM_SSP3

Ref scenario chosen for param: param is GCAM

Diff scenarios chosen for param: param are GCAM_SSP3

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as:

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS.png

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS_DiffAbs.png

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

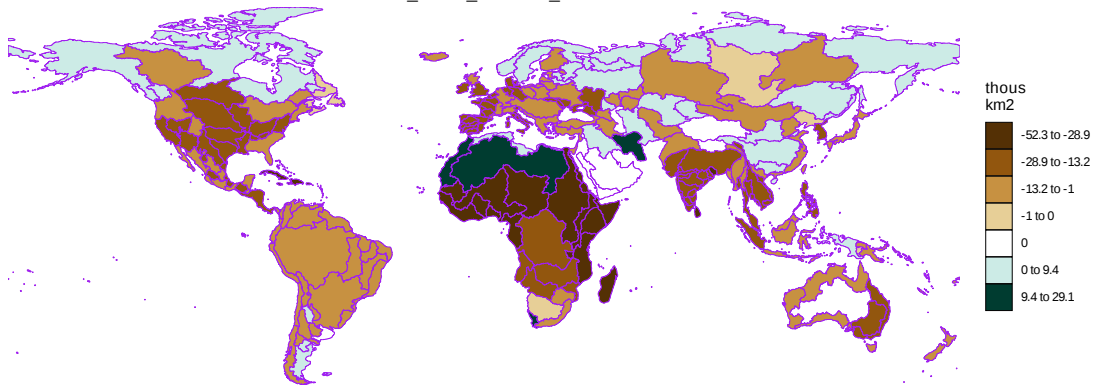
Using map: mapGCAMBasins

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS_DiffPrcnt.png

map run completed.

GCAM_SSP3 diffPrnt GCAM

GCAM_SSP3_DiffPrnt_GCAM



Add a background Setting the background argument to “T” adds a blue background to your maps.

```
[14]: mapx <- map(land_use_forest_2100,  
  show = F,  
  underLayer = "mapGCAMBasins",  
  scenRef = "GCAM",  
  legendType = "kmeans",  
  palette = "pal_green",  
  paletteDiff = "pal_div_BrGn",  
  color = "purple",  
  background = T)
```

```
mapx$map_param_KMEANS_DiffPrct
```

```
options(repr.plot.width = 10, repr.plot.height = 10, repr.plot.res = 100)
```

Starting map...

Running difference against all available scenarios:

GCAM_SSP3

Ref scenario chosen for param: param is GCAM

Diff scenarios chosen for param: param are GCAM_SSP3

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as:

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS.png

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS_DiffAbs.png

Map data table written to

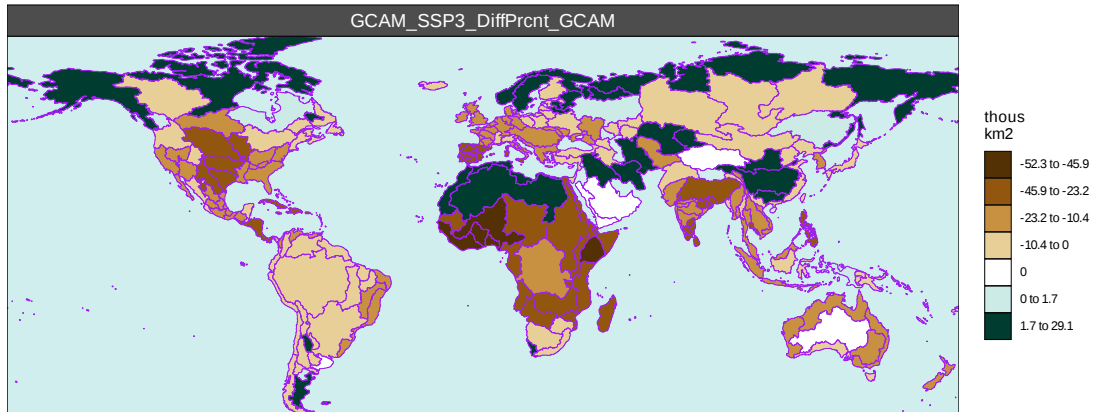
/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

Using map: mapGCAMBasins

Figure saved as: /home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS_DiffPrct.png

map run completed.

GCAM_SSP3 diffPrnt GCAM



Multiple parameters If you want a figure that shows multiple maps for different parameters, like type of forest by scenario, `rmap` can help by using the `row` and `col` arguments.

```
[15]: # modify dataframe
land_use_params <- land_use_rmap %>%
  # filter for crop(s) of interest
  filter(grepl("Forest", landleaf),
    # filter for a year of interest
    year == 2100) %>%
  # separate the landleaf column to get the specific forest types
  separate(landleaf, c("forest1", "forest2", "basin"), sep = "_") %>%
  # do not include protected lands
  filter(!grepl("Protected", forest1)) %>%
```

```

# summarize the dataframe to get aggregate GCAM 32 regional values
group_by(Units, scenario, subRegion, forest1, year) %>%
mutate(value = sum(value)) %>%
distinct(Units, scenario, subRegion, forest1, year, value) %>%
# renaming because rmap does not like columns named "scenario" and "year"
↳ for faceted maps
  rename(scen = scenario,
         yr = year)

# map multiple parameters
mapx <- map(land_use_params,
            show = F,
            underLayer = "mapGCAMReg32",
            legendType = "kmeans",
            palette = "pal_green",
            paletteDiff = "pal_div_BrGn",
            color = "purple",
            background = T,
            row = "scen",
            col = "forest1",
            width = 15,
            height = 10)

```

Starting map...

Map data table written to

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs//map_param.csv

More than one pre-loaded map contain the subRegions in the data provided.

Choosing map based on pre-set map ranking.

	map	subRegNum	rank
1	mapGCAMReg32	32	12.0
2	mapGCAMReg32Uruguay	32	12.5

To choose a different map, please assign it in shape directly.

Using map: mapGCAMReg32

Figure saved as:

/home/joseph.brown@pnnl.go_z38wj6q692/notebooks/outputs/map_param_KMEANS.png

map run completed.

This code let's you view your faceted map - otherwise, you can navigate to the "outputs" folder to view it. Now, you can see changes in forest allocation, by type, between scenarios, in 2100 all in one figure.

